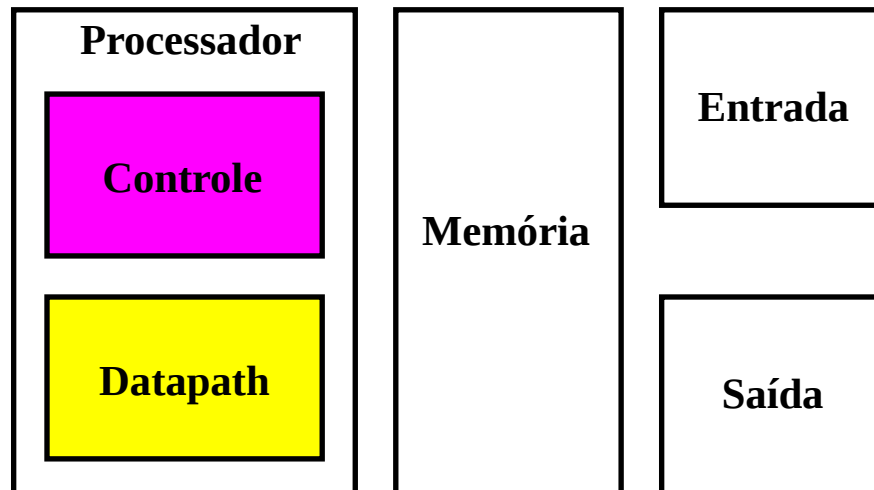


Pipelining avançado – Parte 3



Exploiting subword parallelism

- **Optimized C code: (exploiting compiler intrinsics)**

```
1. #include <x86intrin.h>
2. void dgemm (int n, double* A, double* B, double* C)
3. {
4.     for ( int i = 0; i < n; i+=4 )
5.         for ( int j = 0; j < n; j++ ) {
6.             __m256d c0 = _mm256_load_pd(C+i+j*n); /* c0 = C[i]
               [j] */
7.             for( int k = 0; k < n; k++ )
8.                 c0 = _mm256_add_pd(c0, /* c0 += A[i][k]*B[k][j] */
9.                                     _mm256_mul_pd(_mm256_load_pd(A+i+k*n),
10.                                                    _mm256_broadcast_sd(B+k+j*n)));
11.             _mm256_store_pd(C+i+j*n, c0); /* C[i][j] = c0 */
12.         }
13. }
```

Exploiting subword parallelism

• Optimized x86 assembly code:

```

1. vmovapd (%r11),%ymm0      # Load 4 elements of C into %ymm0
2. mov %rbx,%rcx             # register %rcx = %rbx
3. xor %eax,%eax             # register %eax = 0
4. vbroadcastsd (%rax,%r8,1),%ymm1 # Make 4 copies of B element
5. add $0x8,%rax             # register %rax = %rax + 8
6. vmulpd (%rcx),%ymm1,%ymm1 # Parallel mul %ymm1, 4 A elements
7. add %r9,%rcx              # register %rcx = %rcx + %r9
8. cmp %r10,%rax             # compare %r10 to %rax
9. vaddpd %ymm1,%ymm0,%ymm0  # Parallel add %ymm1, %ymm0
10. jne 50 <dgemm+0x50>       # jump if not %r10 != %rax
11. add $0x1,%esi            # register % esi = % esi + 1
12. vmovapd %ymm0, (%r11)    # Store %ymm0 into 4 C elements

```

[Generated with gcc (2014 version) when using C Intrinsics
to induce full AVX exploitation]

(Inner loop only, i.e. it corresponds to lines 6 to 11 from source code)

Exploiting ILP

• Unrolled C code

```

1 #include <x86intrin.h>
2 #define UNROLL (4)
3
4 void dgemm (int n, double* A, double* B, double* C)
5 {
6   for ( int i = 0; i < n; i+=UNROLL*4 )
7     for ( int j = 0; j < n; j++ ) {
8       __m256d c[4];
9       for ( int x = 0; x < UNROLL; x++ )
10        c[x] = _mm256_load_pd(C+i*x*4+j*n);
11
12       for( int k = 0; k < n; k++ )
13       {
14         __m256d b = _mm256_broadcast_sd(B+k*j*n);
15         for (int x = 0; x < UNROLL; x++)
16          c[x] = _mm256_add_pd(c[x],
17                               _mm256_mul_pd(_mm256_load_pd(A+n*k+x*4+i), b));
18       }
19
20       for ( int x = 0; x < UNROLL; x++ )
21        _mm256_store_pd(C+i*x*4+j*n, c[x]);
22     }
23 }

```

Matrix Multiply

• Assembly code:

```

1 vmovapd (%r11),%ymm4      # Load 4 elements of C into %ymm4
2 mov %rbx,%rax             # register %rax = %rbx
3 xor %ecx,%ecx             # register %ecx = 0
4 vmovapd 0x20(%r11),%ymm3   # Load 4 elements of C into %ymm3
5 vmovapd 0x40(%r11),%ymm2   # Load 4 elements of C into %ymm2
6 vmovapd 0x60(%r11),%ymm1   # Load 4 elements of C into %ymm1
7 vbroadcastsd (%rcx,%r9,1),%ymm0 # Make 4 copies of B element
8 add $0x8,%rcx # register %rcx = %rcx + 8
9 vmulpd (%rax),%ymm0,%ymm5  # Parallel mul %ymm1,4 A elements
10 vaddpd %ymm5,%ymm4,%ymm4   # Parallel add %ymm5, %ymm4
11 vmulpd 0x20(%rax),%ymm0,%ymm5 # Parallel mul %ymm1,4 A elements
12 vaddpd %ymm5,%ymm3,%ymm3   # Parallel add %ymm5, %ymm3
13 vmulpd 0x40(%rax),%ymm0,%ymm5 # Parallel mul %ymm1,4 A elements
14 vmulpd 0x60(%rax),%ymm0,%ymm0 # Parallel mul %ymm1,4 A elements
15 add %r8,%rax              # register %rax = %rax + %r8
16 cmp %r10,%rcx            # compare %r8 to %rax
17 vaddpd %ymm5,%ymm2,%ymm2   # Parallel add %ymm5, %ymm2
18 vaddpd %ymm0,%ymm1,%ymm1   # Parallel add %ymm0, %ymm1
19 jne 68 <dgemm+0x68>       # jump if not %r8 != %rax
20 add $0x1,%esi            # register %esi = %esi + 1
21 vmovapd %ymm4, (%r11)     # Store %ymm4 into 4 C elements
22 vmovapd %ymm3, 0x20(%r11) # Store %ymm3 into 4 C elements
23 vmovapd %ymm2, 0x40(%r11) # Store %ymm2 into 4 C elements
24 vmovapd %ymm1, 0x60(%r11) # Store %ymm1 into 4 C elements

```

[Generated with gcc (2014 version) when using C Intrinsics for full **AVX** exploitation and **03**]

Performance Impact

